

Go WaitGroups — What the Video Teaches

The video covers a real problem with goroutines and how WaitGroups solve it.

The Problem

When you launch goroutines, the main program doesn't wait for them to finish. It just exits. The "hacky" fix is using `time.Sleep()` to delay the exit — but in real-world code, you don't know exactly how long goroutines will take, so this approach doesn't work.

The Solution: WaitGroup

A WaitGroup is a **synchronization mechanism** from the `sync` package. Think of it as a **counter**:

- When a goroutine starts → **increment** the counter (`Add(1)`)
- When a goroutine finishes → **decrement** the counter (`Done()`)
- Main program **blocks** at `Wait()` until the counter hits zero, then exits

Three things to remember:

`wg.Add(1)` → goroutine is starting
`wg.Done()` → goroutine has finished
`wg.Wait()` → block here until all goroutines are done

The Code Pattern Explained

```
var wg sync.WaitGroup

for ... {
    wg.Add(1)          // one more goroutine starting
    go doTask(&wg)      // pass pointer of wg to the function
}
```

```
wg.Wait()           // main blocks here until counter = 0

func doTask(wg *sync.WaitGroup) { // receives pointer, not value
    defer wg.Done()              // runs at end of function automatically
    // ... actual work
}
```

Key Points the Instructor Emphasizes

Why pointer (sync.WaitGroup*)?** You must pass the WaitGroup as a pointer, not a value copy. If you pass by value, the `Done()` call won't affect the actual counter — it'll decrement a copy nobody is watching.

Why *defer wg.Done()*? *defer* means "run this at the very end of the function, no matter what." So even if the function panics or returns early, `Done()` is guaranteed to be called. The instructor compares it to React's *useEffect* cleanup function — similar concept.

Why pass *wg* into the function at all? Because *defer wg.Done()* needs to live *inside* the goroutine function. If you're using an **anonymous/inline function** instead, you don't need to pass it — you can directly access *wg* from the outer scope and call *Add* and *Done* there itself.

How the Counter Works (with 2 goroutines)

```
Start:    counter = 0
Goroutine 1 launches → Add(1) → counter = 1
Goroutine 2 launches → Add(1) → counter = 2
Goroutine 1 finishes → Done() → counter = 1
Goroutine 2 finishes → Done() → counter = 0
                ↓
            wg.Wait() unblocks → program exits
```

The instructor ends by saying you'll see WaitGroups **everywhere** in real Go projects, so make sure you actually understand this rather than just copying the pattern.